

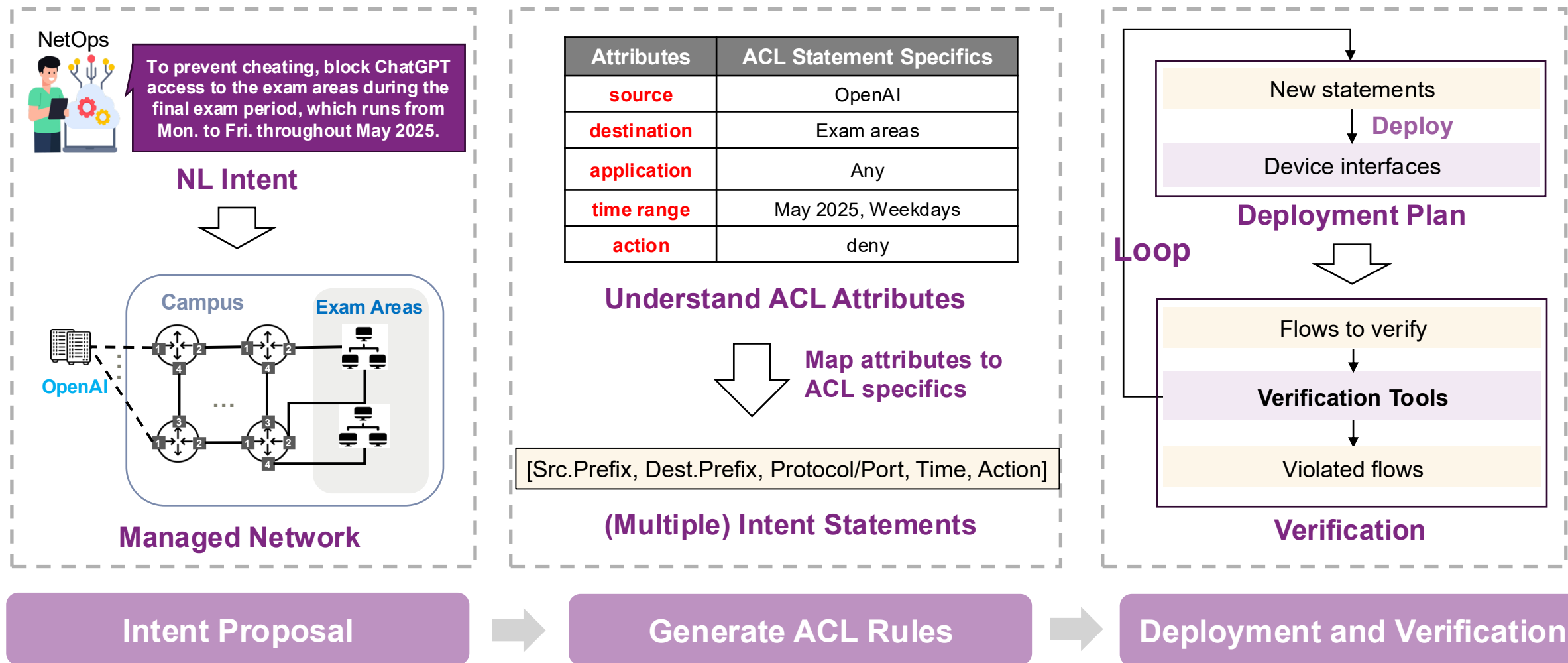


Automating ACL Configuration with Natural Language Intents

Wenlong Ding

2025.2.19

Common Practice of Manual ACL Configuration



- Both existing literature (Alibaba) and operational experience (Azure) reveal this process is labor-intensive and can take up to a week.
- End-to-end automation with minimal NetOps intervention is needed!

The automation of the ACL configuration process presents three key challenges:

Reasoning from Intent to Statement

- Complex ACL syntax (rule format) and network details (prefixes, interfaces, etc.)
- A single intent may map to multiple ACL statements configured at different device interfaces

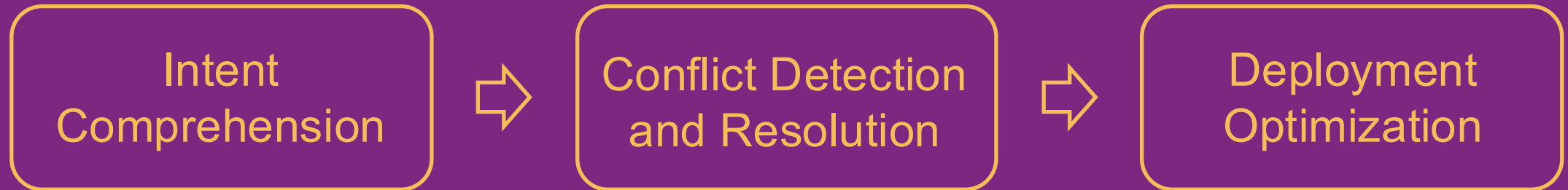
Detecting Conflicts with Existing Rule

- Intent and existing ACL statements handle overlapping flows with conflicting policy actions.
- Manual check is impractical: $O(100)$ backbone routers in WAN and $O(1000)$ existing ACL rules per router interface.

Optimizing the Deployment

- NetOps prefer minimal new statement additions to simplify configuration maintenance and subsequent updates.
- Traditional methods of deploying new statements across all routing paths or gateway interfaces are redundant.

Xumi (須弥): Automatic ACL Configuration with Natural Language Intents



香港中文大學
The Chinese University of Hong Kong



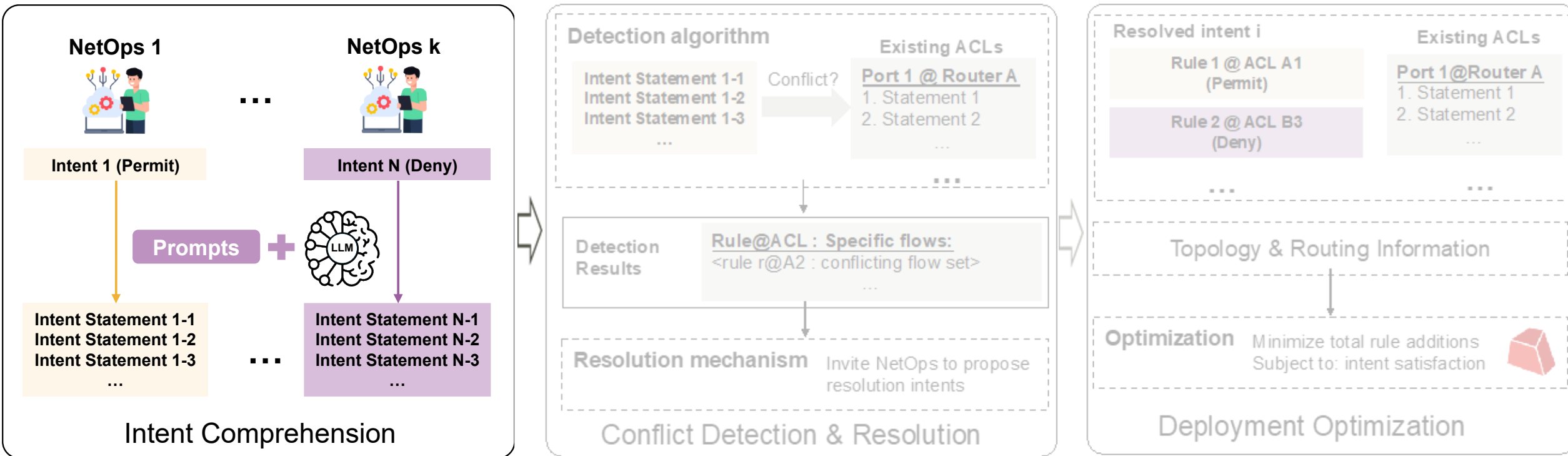
Microsoft



THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)



Task 1: Intent Comprehension



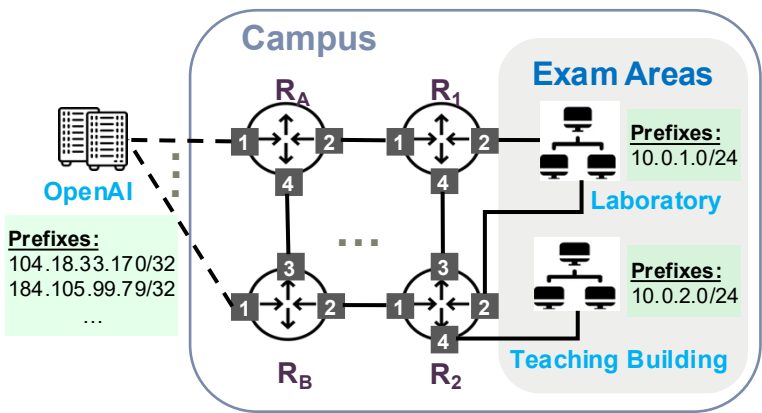
LLM Issue 1: Instilling Network-Specific Information



Network-Specific Information is Not in Intents or LLM General Knowledge

- *Endpoint prefixes* for ACL statement
- *Gateway interfaces* for deployment

Solution: Semantics-Network Mapping Table (SNMT) for LLM



Network

NetOps
Maintenance

Up-to-Date
Information

Endpoint	Gateway Router @ Interface	Prefixes
Laboratory	$R_1@2, R_2@2$	10.0.1.0/24
Teaching Building	$R_2@4$	10.0.2.0/24
Exam Areas	$R_1@2, R_2@2$	10.0.1.0/24
	$R_2@4$	10.0.2.0/24
OpenAI	$R_A@1, R_B@1$	104.18.33.170/32, 184.105.99.79/32, ...

SNMT

Map endpoints to their gateway devices, interfaces and prefixes.

Operators can register only one or a group of endpoints.



Hallucinations Cause Incorrect Results

- *Missing Details*: Lack of prefixes or protocols if there are multiple ones
- *Non-Existent Element*: Fabricate an entity that does not exist
- ...

Solution: Hallucination Mitigation Prompts

1. Few-Shot Demonstration (in system prompt)

- Correct intent-IR reasoning examples for corner cases
- Example: Describe an unusual dual-protocol case where DNS uses both UDP and TCP

2. Self-Reflection (in system prompt)

- Verification prompts for typical hallucination cases
- Example: Constraint output entity names to be within SNMT

3. NetOps' Feedback (separate from system prompt)

- Final fail-safe mechanism for operators to check and feedback on errors
- Re-prompt LLM with feedback and error output for self-correction

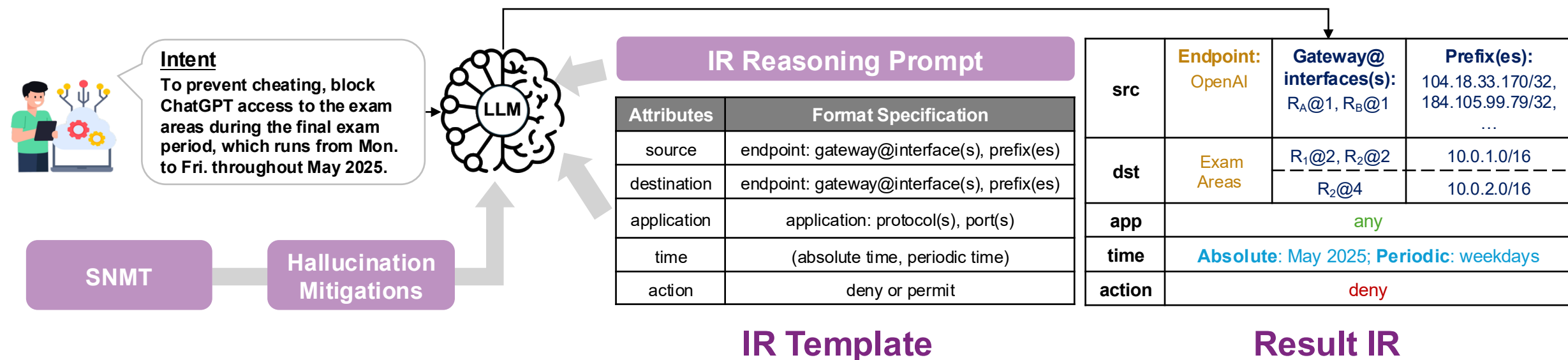


Functions of IR Template

- Define ACL/network specifics to be comprehended
- Define the output format of LLM

IR Reasoning Prompt (in system prompt)

- A step-by-step guidance prompt for LLM to locate required specifics in NL and SNMT



Generate Multiple Statements from IR



Result IR Enumerations

- Enumerate combinations of src/dst prefixes and applications
- Assign src/dst gateway interfaces to each statement based on prefixes
 - Different statements may have different <src, dst> gateway interface pairs

src	Endpoint: OpenAI	Gateway@ interface(s): R _A @1, R _B @1	Prefix(es): 104.18.33.170/32, 184.105.99.79/32, ...
dst	Exam Areas	R ₁ @2, R ₂ @2	10.0.1.0/16
		R ₂ @4	10.0.2.0/16
app	any		
time	Absolute: May 2025; Periodic: weekdays		
action	deny		

Result IR



Intent ACL Statements:

S1: 104.18.33.170/32, 10.0.1.0/24, any, 2025/05:weekdays, deny

S2: 104.18.33.170/32, 10.0.2.0/24, any, 2025/05:weekdays, deny

S3: 184.105.99.79/32, 10.0.1.0/24, any, 2025/05:weekdays, deny

S4: 184.105.99.79/32, 10.0.2.0/24, any, 2025/05:weekdays, deny

...

<src, dst> Gateway Interfaces of Statements:

S1: src: [R_A@1, R_B@1], dst:[R₁@2, R₂@2]

S2: src: [R_A@1, R_B@1], dst:[R₂@4]

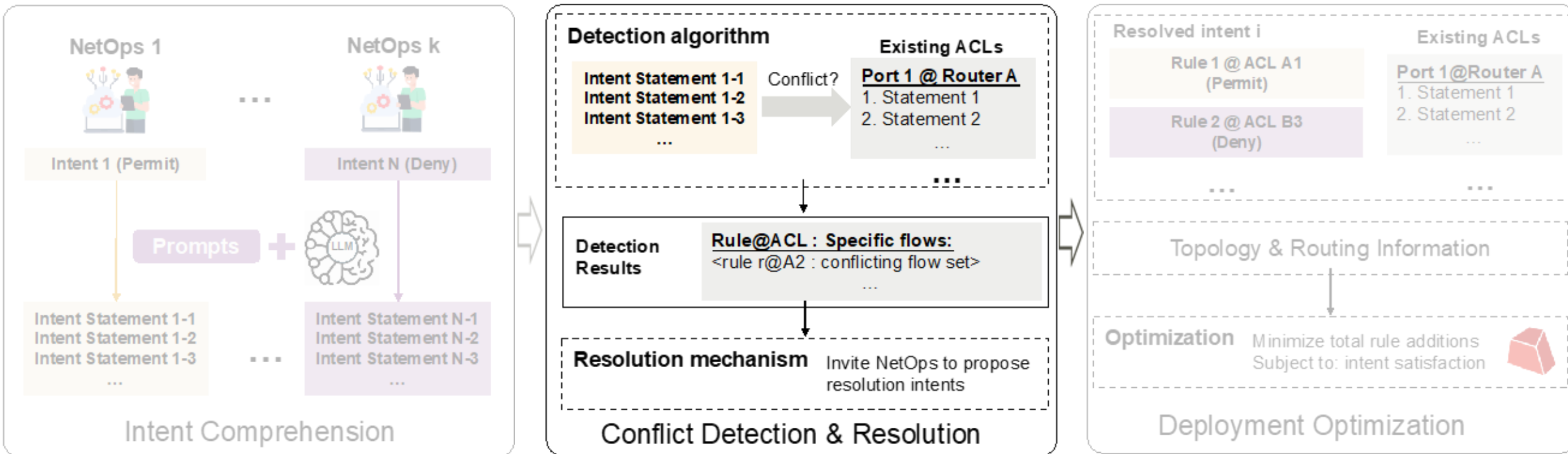
S3: src: [R_A@1, R_B@1], dst:[R₁@2, R₂@2]

S4: src: [R_A@1, R_B@1], dst:[R₂@4]

...

Multiple Result Statements &
Related Endpoint Interfaces

Task 2: Conflict Detection and Resolution



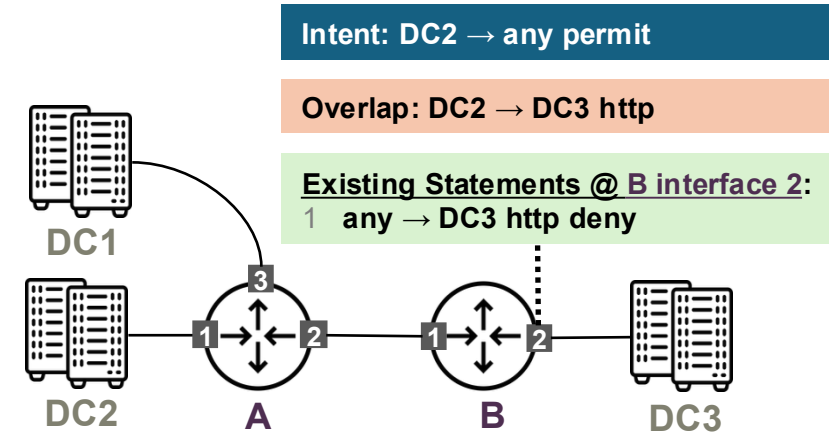
~~Conflict Between Intents~~

- **Assumption:** Operators resolve intent-level conflicts and input conflict-free intents for Xumi

Conflict Between Intent and Existing Statements

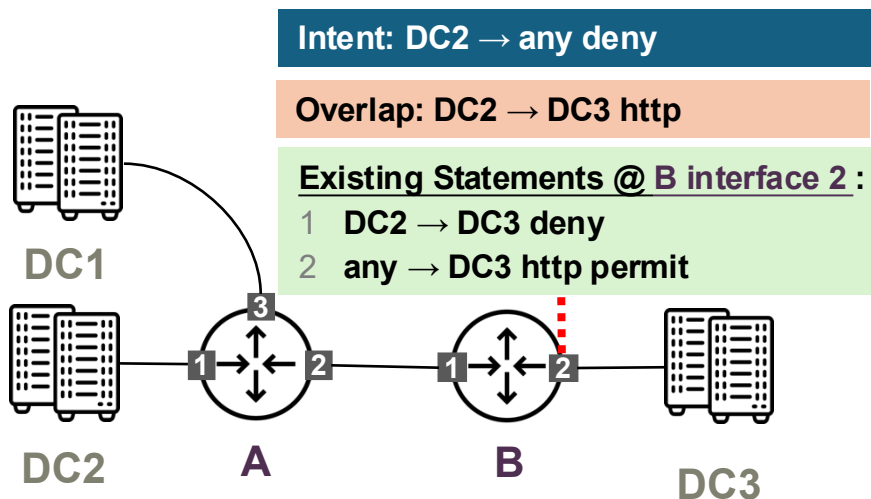
Two conditions should be satisfied simultaneously:

- Overlapping flows
- Opposite actions



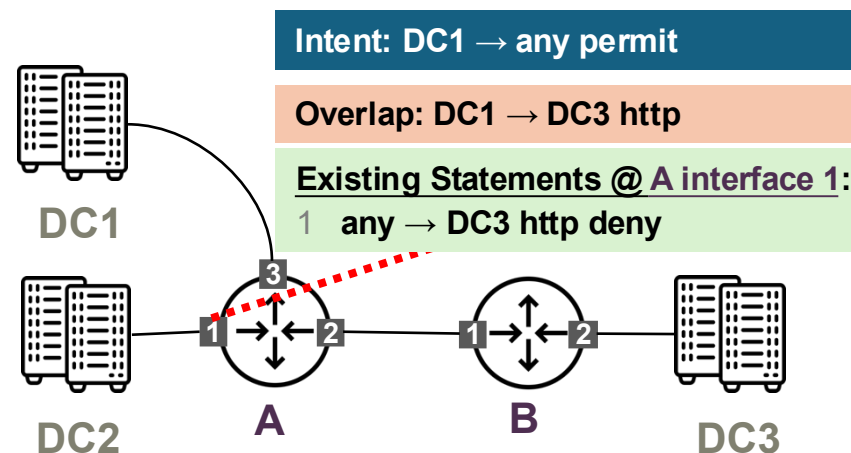
The overlapping flow “DC2→DC3 http” between intent statement “DC2→any permit” and existing rule “any→DC3 deny” is the conflict flow.

False-Positives (FPs) in Conflicts



FP1: Preceding Statements

The overlapping flow “DC2→DC3 http” between intent and 2nd rule is actually managed by a higher priority (1st) rule, and this rule does not cause conflict.



FP2: Infeasible Path

The overlapping flow “DC1→DC3 http” between intent and 1st rule only route on path <A3,A2,B1,B2>, and this path do not traverse interface A₁.

Idea: Perform detection on the **truly matched flow of a statement**

Step 1: Truly-matched flow calculation

- For statement S_k with priority k with specified flow R_k , calculate its truly matched flow T_k :
 - Exclude all higher priority flows R_i ($i < k$)

$$T_k = R_k - \cup_{i=1}^{k-1} T_i$$

- Calculate T_k from the 1st rule initialized with $T_1 = R_1$, storing intermediate $\cup_{i=1}^{k-1} T_i$. (DP calculation process)
- Maintain truly-matched flows for all statements on each interface

Step 2: Conflict detection

- Overlap intent statement with T_k (instead of R_k)
- Apply for all existing statements with anti-intent action

Idea: Verify if the conflict port is on any feasible paths

→ **Step 1: Feasible path detection**

- Check source and destination gateway interfaces of a conflict flow
 - Match src/dst prefixes in SNMT to obtain gateway interfaces
- Enumerate all paths between src-dst gateway interfaces

→ **Step 2: Conflict port verification**

- Verify if the conflict port exists in any enumerated path.
- If no, it is an FP conflict

Resolution with “Protect” Intents



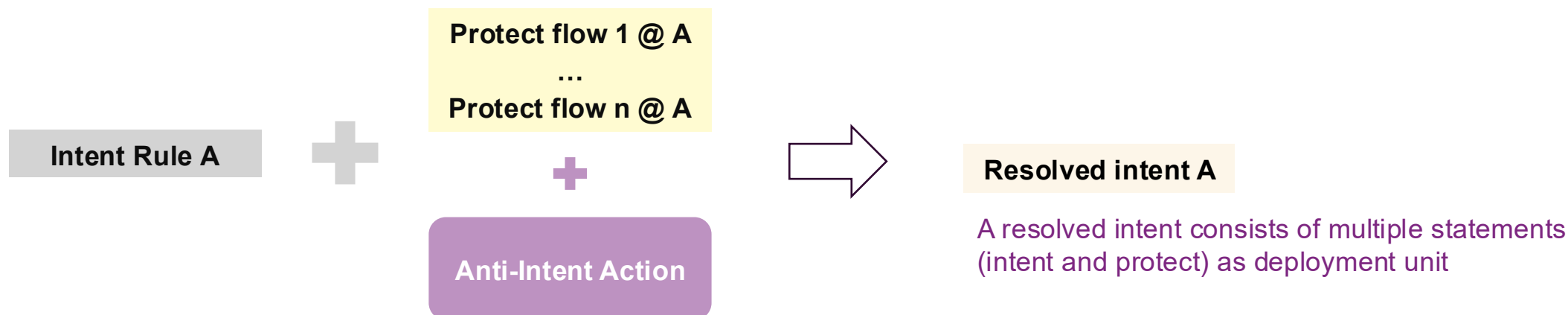
NetOps choose to “protect” certain conflict flows that must follow existing rules

- Xumi assumes “protect” flows as subset of the detected conflict flows
- Example: For conflict “DC1→any”, propose “Protect SSH traffic from DC1”

“Protect” Intent Comprehension

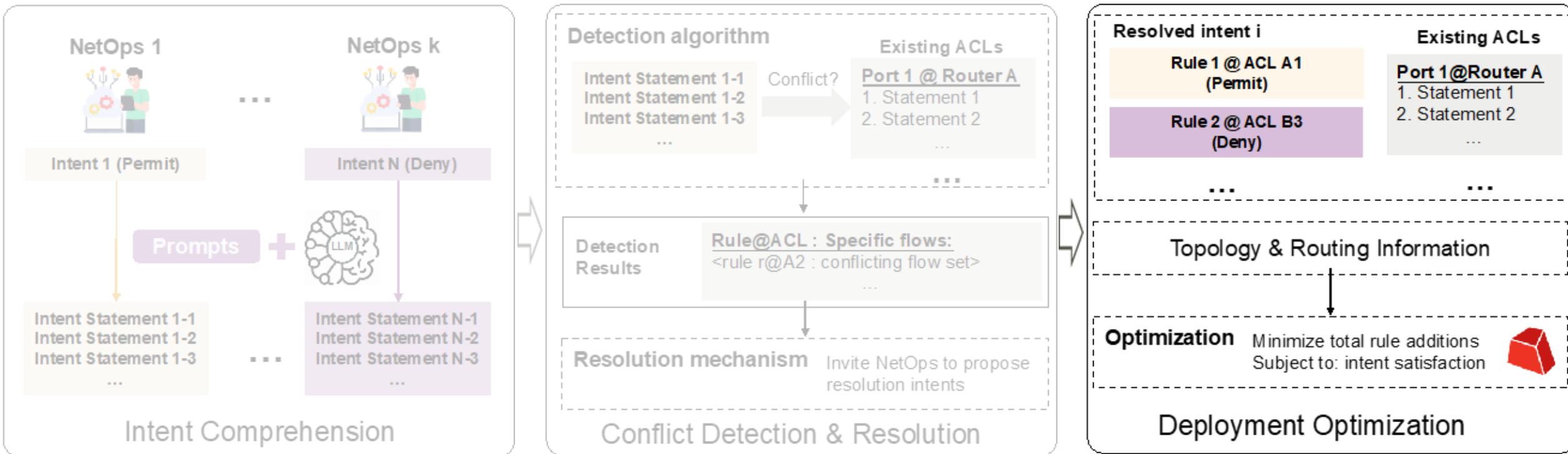
- **Goal:** [source, destination, application, time range] quadruple
- **Method:** Follow *Intent comprehension*, but omit “action” attribute in IR

Resolved Intent Generation





Task 3: New Statement Deployment



Deny Intents

- Deployment constraint: “Deny” rule should exist in at least one interface for each path
- Solution: *Full endpoint interface deployment*
 - Deploy each “deny” statement at all source or destination gateway interfaces

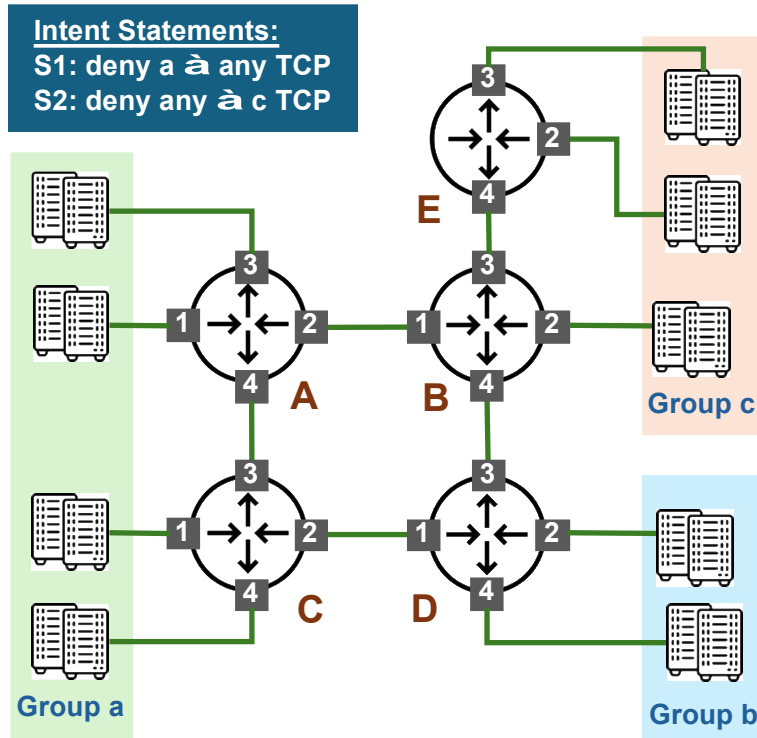
Permit Intents

- Deployment constraint: “Permit” rule should exist in all device interfaces that the statement flow traverse
- Solution: *Full conflict interface deployment*
 - Deploy each “permit” statement on all interfaces that have detected conflicts

Optimization: Redundant Statements



Observation 1: Bottleneck Deployment can Use Fewer Rules/ACLs



Statement $\rightarrow$ Deployment interfaces

S1 $\rightarrow$ A1, A3, C1, C4

S2 $\rightarrow$ E2, E3, B2

Full deployment (Naive)

Statement $\rightarrow$ Deployment interfaces

S1 $\rightarrow$ B1, D1

S2 $\rightarrow$ B1, B4

Bottleneck deployment

7 statements

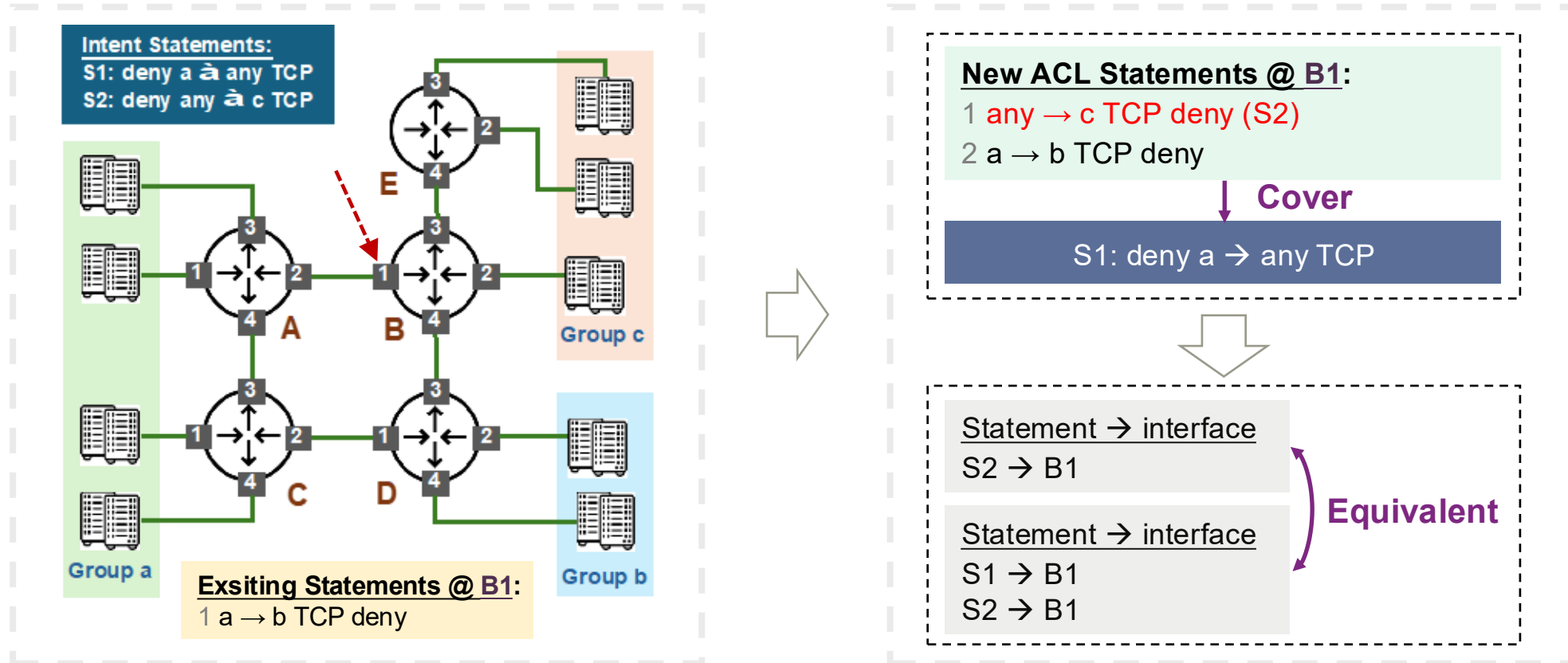
4 statements

Deploy the statement on bottleneck interfaces that multiple paths route through

Optimization: Redundant Statements



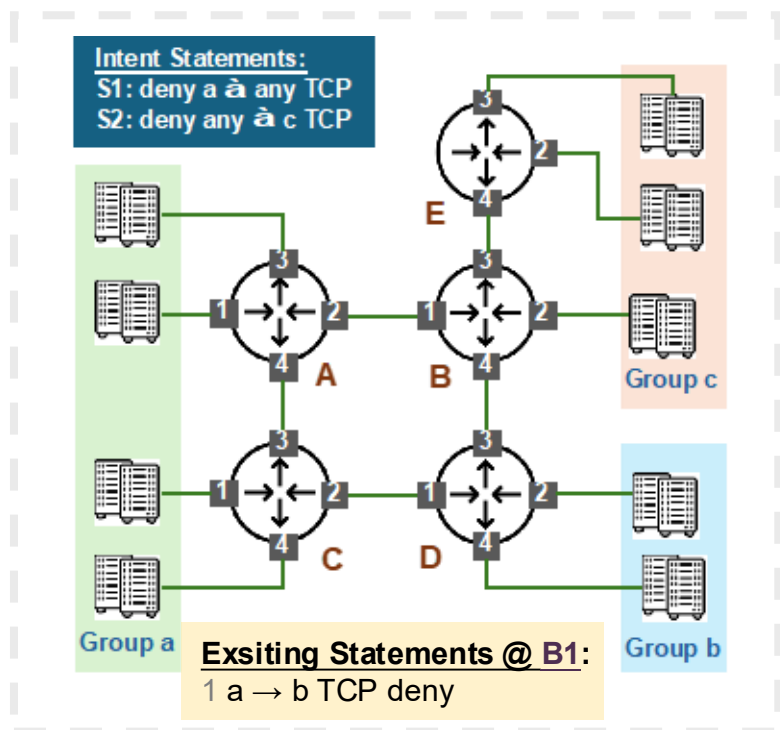
Observation 2: Effect-Equivalent Deployment: One Stone Kills Two Birds!



Optimization Observations: Reduce Redundant Statements



Observation 2: Effect-Equivalent Deployment (“Deny” Example)



Bottleneck deployment

Statement $\rightarrow$ Deployment interfaces

S1 $\rightarrow$ B1, D1

S2 $\rightarrow$ B1, B4

Equivalent

S1 $\rightarrow$ B1
S2 $\rightarrow$ B1

S2 $\rightarrow$ B1

Statement $\rightarrow$ Deployment interfaces

S1 $\rightarrow$ D1

S2 $\rightarrow$ B1, B4

Effect-equivalent Deployment

4 statements

3 statements

- This observation also holds for “permit” statements

- Key question: Whether deploying resolved intent RI_j on interface p can achieve the same effect of deploying resolved intent RI_i with complementary existing rules on p ?
 - Equivalence calculation scope for RI_i
 - The group of resolved intent $(RI_j, \forall j)$ with the *same* original intent action
 - Interface p on RI_i ' routing paths
 - RI_i has deployment potential on p (existing rules on p do not contain RI_i , i.e., has conflict flows)v
 - Equivalence Condition: RI_j^{flow} + TMF of existing rules with same action **cover** RI_i^{flow}

$$E_{pi} = \{RI_j | RI_i^{flow} \subseteq \left(\bigcup_k^{T_{pk}.act=RI_j.act} T_{pk} \cup RI_j^{flow} \right), \forall j\}$$

- E_{pi} : equivalent intent set of RI_i on interface p

Design: Optimization Formulation



N_{pi} : Whether deploy resolved intent i on interface p

t_i : statement number of resolved intent i

E_{pi} : equivalent statement set of resolved intent i on interface j

C_{pi} : Conflict set of original intent statement i on interface j

Ω_i : path set of intent i

Permit optimization (Interface level)

$$\min \sum_p \sum_i t_i N_{pi}$$

$$\text{s.t. } \forall j \in E_{pi} N_j = 1, \\ \forall i, p \text{ such that } C_{pi} \neq \emptyset$$

Each (resolved) intent can be implemented using any of its equivalents.

Objective: Minimize total rule additions across all interfaces

Deny optimization (global network)

$$\min \sum_p \sum_i t_i N_{pi}$$

$$\text{s.t. } \forall p \in P \forall j \in E_{pi} N_{pj} = 1, \\ \forall i, P \in \Omega_i \text{ such that } \bigwedge_{p \in P} (C_{pi} \neq \emptyset) = 1$$

Each path needs to be deployed only once, and on every interface, intent can be replaced with its equivalent.

Evaluations

Overview: End-to-End Performance Highlight

- Complete configuring 10 intents within 10 minutes for modern-scale cloud WAN (171 routers)
 - Much faster than traditional human effort (1 week)
 - 10x faster than a baseline (resemble NetConfEval, Lumi) also using LLMs for intent comprehension but ignoring conflicts and deployment optimization.
- Addressing nearly 100 ACLs
- Reduce over 99% new rule additions compared to current practice (deploy on all interfaces on paths)

Intent Comprehension



Evaluated networks

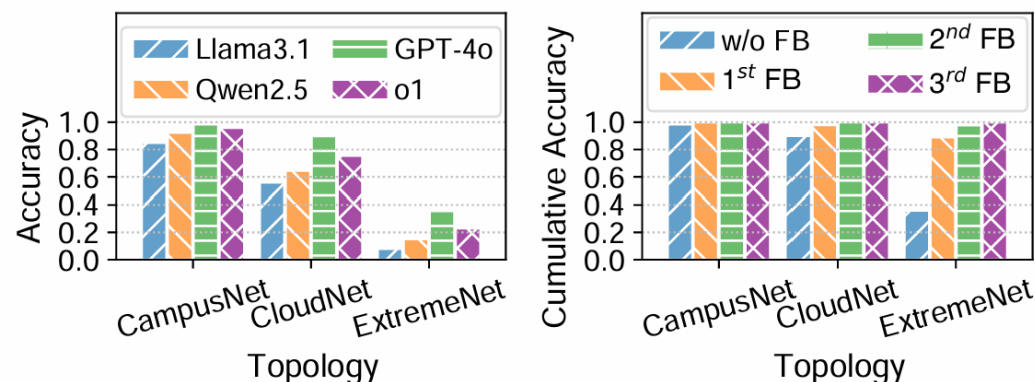
	Network Information				#Prompt Tokens (SNMT ratio)
	#Routers	#Links	#Interfaces	#Entities	
<i>CampusNet</i>	41	66	361	60	4407 (51%)
<i>CloudNet</i>	171	219	2524	352	38755 (94%)
<i>ExtremeNet</i>	1026	1236	11796	2062	178414 (99%)

- CampusNet: Mimic CUHK's campus network.
- CloudNet: Mimic Azure's global cloud WAN.
- ExtremeNet: 6x Azure's WAN.

Time

Time (s)	Initial Inference Time w/o Feedback				Cumulative Feedback Time (GPT-4o)			
	Llama3.1-70B	Qwen2.5-72B	GPT-4o	OpenAI o1	w/o FB	1 st FB	2 nd FB	3 rd FB
<i>CampusNet</i>	3.8	3.2	2.5	4.7	2.5	82.3	-	-
<i>CloudNet</i>	8.7	6.9	5.2	11.4	5.2	71.7	96.0	-
<i>ExtremeNet</i>	9.1	14.4	11.3	16.9	11.3	152.8	267.8	370.8

Accuracy



(a) Accuracy w/o Feedback

(b) Feedback Accuracy (GPT-4o)

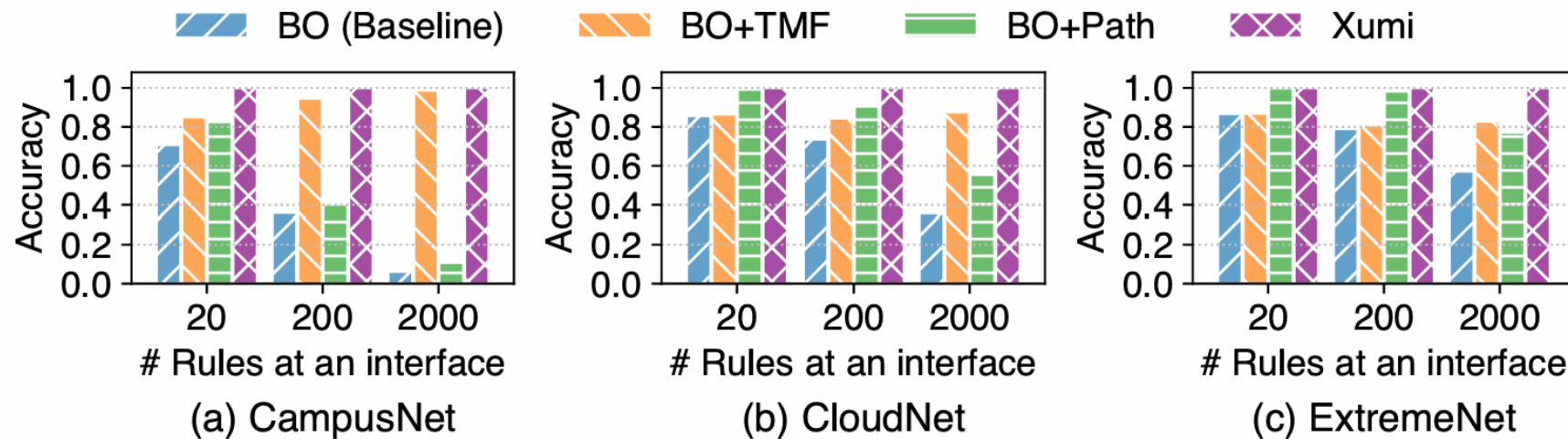
- Achieve 98.5% and 90% accuracy with GPT-4o without any NetOps feedback for modern campus and cloud network.
- All errors can be corrected within 3 rounds of NetOps feedback.

- Achieve 100% accuracy within 8 minutes.

Compared Schemes:

- Blind Overlapping (BO) (Baseline)
- BO + Truly-Matched-Flow (TMF): Considering FP1 only
- BO + Path: Considering FP2 (routing paths) only

Accuracy

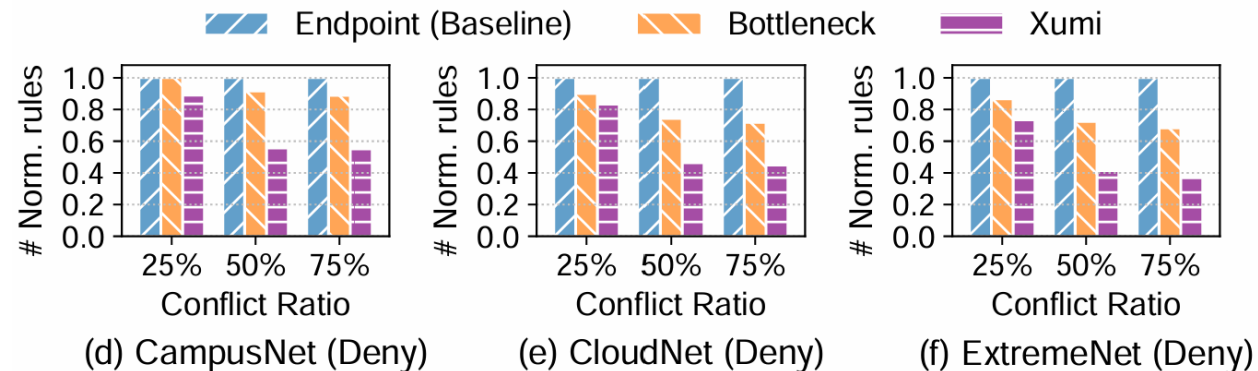
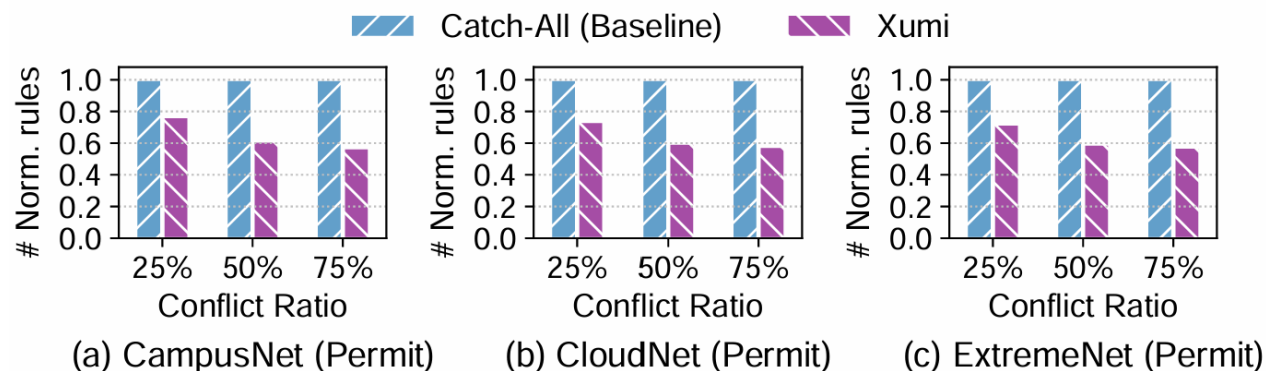


- **Accuracy:** 3.3x higher than baseline method ignoring all FPs on average.
- **Time:** Within 10 minutes on average. Within 2 hours for the largest network with 10k interfaces and 2k rules per interface.

Compared Schemes (Permit):

- Catch-All: deploy on all interfaces that have conflict

Normalized statement additions



Compared Schemes (Deny):

- Endpoint: deploy on source/destination gateway interfaces
- Bottleneck: Only consider optimization of bottleneck interface observation.

- Rule Additions:** reduce 38.8% on average
 - Up to 60% for the largest ExtremeNet
- Time:** Fast solving speed within 3 minutes for all settings.



Conflict Detection Acceleration

- BitArray-based flow set operations (intersection, difference, union, subset...).
- Early termination.

Deployment Optimization Acceleration

- Pre-calculate the accumulative TMF for “permit” and “deny” on all interfaces for equivalent intent set checking.
- Logical constraints (instead of binary multiplication) and parallel computing for optimization formulation with Gurobi.



Thank You!

Discussion



WELCOME!